

Topic: Structures with Pointers — Self-Learning

Q. Explain the concept of Structures in C. How can we use pointers with structures? Provide a code example to demonstrate its use.

📌 Definition to Remember

A **Structure** (`struct`) in C is a user-defined data type that groups variables of **different data types** under a single name. A **Pointer to a Structure** stores the memory address of a structure, enabling **dynamic memory allocation** and efficient function passing. Structure members are accessed via a pointer using the **Arrow Operator** (`→`).

1. Structures in C

- Groups multiple variables (called **members/fields**) of different types into one logical unit.
- Ideal for representing real-world entities (Student, Employee, etc.).
- **Syntax:**

```
struct Student {  
    int roll_no;  
    char name[30];  
};
```

- Access members using the **dot operator** (`.`): `s.roll_no`

2. Pointers with Structures

When a structure is passed to a function **by value**, a full copy is made — wasteful for large structures. Passing a **pointer to the structure** is more efficient.

- **Arrow Operator** (`→`) is used to access members via a pointer.
- `ptr→member` is equivalent to `(*ptr).member`

3. Self-Referential Structure

A structure that contains a **pointer to another structure of the same type** — the building block of **Linked Lists and Trees**.

```
struct Node {  
    int data;  
    struct Node *next;    // Points to the next node of same type  
};
```

4. Complete Code Example

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct Student {
    int roll_no;
    char name[30];
};

int main() {
    // Dynamic memory allocation using pointer to structure
    struct Student *ptr = (struct Student *)malloc(sizeof(struct Student));

    if (ptr == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    // Access members using Arrow Operator (→)
    ptr→roll_no = 101;
    strcpy(ptr→name, "John Doe");

    printf("Student Name: %s\n", ptr→name);
    printf("Roll Number: %d\n", ptr→roll_no);

    free(ptr); // Release allocated memory
    return 0;
}

```

5. Dot vs Arrow Operator

Situation	Operator	Syntax
Normal variable	Dot (.)	s.roll_no
Pointer to structure	Arrow (→)	ptr→roll_no

★ Must-Write Points (for 10 marks)

1. **struct** groups variables of different types under one name.
2. Members accessed using dot (.) for normal variables; Arrow (→) for pointers.
3. Passing a pointer to a function avoids copying the entire structure — more efficient.
4. Dynamic memory: use `malloc(sizeof(struct Name))` to allocate structure at runtime.
5. `ptr→member` is equivalent to `(*ptr).member`.
6. **Self-Referential Structure**: contains a pointer to itself — used to build Linked Lists and Trees.
7. Always `free(ptr)` after dynamic allocation to prevent memory leaks.

⚡ Quick Recall

struct → Groups different types → Dot (.) for variable → Arrow (→) for pointer → malloc (dynamic) → Self-Referential (Linked List/Tree) → free() to release